

# GPU-Accelerated LIF Spiking Neural Networks Simulator

Emre Sagir

Computer Engineering (GPU Programming Class)

Politecnico di Torino

Torino, Italy

s339049@studenti.polito.it

**Abstract**—This paper presents a GPU-Accelerated simulator that is based on a discrete-time LIF (Leaky Integrate and Fire) neuron model and a synaptic model. Simulator is written in C and CUDA, and validated against the Torch reference to ensure correctness. Four models are created in total. C and Torch versions, which run on the CPU; Basic and Optimized versions for the GPU. The optimized version consists of three different optimization techniques: Kernel fusion, shared memory tiling, and vectorized load. From the results, the optimized GPU version achieved 3.3x speed-up compared to the basic GPU version, 112x compared to the CPU version. When the neuron count got larger, the optimized and basic GPU versions converged on the speed-up graph; this is because the models hit the memory-bandwidth limit.

**Index Terms**—spiking neural networks, leaky integrate-and-fire, GPU programming, CUDA, synaptic decay, simulator

## I. BACKGROUND

### A. Neuron Model

The discrete-time implementation of the Leaky Integrate-and-Fire (LIF) model as discussed by Stan and Rhodes [1] allows for efficient sequence modeling in SNNs.

The parameters in the equations are:

- $u[t]$  Membrane voltage (potential) at time step  $t$ .
- $\beta$  Membrane decay factor.
- $s[t]$  Spike indicator.
- $\theta$  Firing threshold voltage.
- $i[t]$  Input current.

The membrane potential update is being done with the equation 1.

The Reset mechanism is given in equation 2 as soft reset. Instead of resetting to a default value, the membrane voltage is being calculated directly. This prevent warp divergence if the hard reset chosen, because the if condition structure will be introduced.

Lastly, spike generation is being calculated with equation 3. The spike is being decided with comparison between membrane voltage and threshold.

$$u[t] = \beta u[t-1] + (1 - \beta)i[t] \quad (1)$$

$$u[t] \leftarrow u[t] - s[t-1]\theta \quad (2)$$

$$s[t] = \Theta(u[t] - \theta) = \begin{cases} 1, & \text{if } u[t] > \theta \\ 0, & \text{otherwise} \end{cases} \quad (3)$$

where the decay factor  $\beta$  is defined by the membrane time constant  $\tau$  and simulation time step  $\Delta t$ :

$$\beta = \exp\left(-\frac{\Delta t}{\tau}\right) \quad (4)$$

### B. Synapse Model

Firstly, the SNN simulator could have been without the synapse model. The synapses could have been just the weights without the leakage. But for the sake of showing the power of the GPU and increasing complexity, the synapse model was implemented.

The synapse model is decided as current-based exponential synapse model. The equation of the exponential synapse model is given in the equation 5.

Parameters are given for synapse model:

- $\alpha$  Synaptic decay factor ( $e^{-\Delta t/\tau_s}$ ).
- $w_{ji}$  Weight from neuron  $j$  to  $i$ .
- $s_j[t]$  Spike indicator from presynaptic neuron  $j$ .
- $g_i[t]$  Postsynaptic current state (decaying memory).
- $u_i[t]$  Membrane potential of neuron  $i$ .
- $\beta$  Membrane decay factor ( $e^{-\Delta t/\tau_m}$ ).
- $\theta$  Spike threshold.

$$g_i[t] = \alpha g_i[t-1] + \sum_j w_{ji} s_j[t] \quad (5)$$

Connecting this synapse model with the neuron model is resulted this equation 6. The input current is changed with the Postsynaptic current state, which is calculated in equation 5.

$$u_i[t] = \beta u_i[t-1] + (1 - \beta)g_i[t] \quad (6)$$

Current implementation of the model is dense. The sparsity can be added later, but will introduce a high level of complexity.

## II. PROPOSED METHOD

From the previous section, we know the spiking neural network calculation depends on repeatedly performing the given calculations over a specified number of timesteps. Even though different timesteps have data dependencies, different neurons can be calculated on different threads, which gives us the opportunity to exploit the GPU's parallel computing power.

My approach basically involves having different models do the same work, and comparing their performance with different parameters (neuron count, timestep).

Firstly, creating a base with the CPU Model, then having a reference for its correctness by using the Python torch library. Secondly, creating a basic GPU model, which will give us a baseline for the GPU. Lastly, optimizing the GPU Model. This process will provide four different models that can be compared.

Every model will be compared with a small test set to be sure that all the calculations are correct.

At the end, the performance comparison will be done between the models to understand the speed-up. And will compare the work with other related works.

## III. CPU MODEL

### A. Implementation

The CPU implementation is done with the given neuron and synapse model definitions in the previous chapters. It is written in C. The main time loop is given in Figure 1.

For the synaptic update, the external input current is calculated with the given external spikes and weights, then, with a second for loop, the recurrent connections are accumulated into the input current; lastly, the synaptic decay is being applied.

For the neuron update, a soft reset is applied if there was a spike in the previous timestep; then the leakage and integration of the input current are applied. Lastly, a spike is recorded if the membrane potential is higher than the threshold.

### B. Testing

To check the correctness of the C model, Pytorch version is also implemented with the same model definitions. The comparison test is done.

Firstly, the Poisson (input) spikes are saved while running the PyTorch version. Then those spikes are imported to the C version as can be seen in the Figure 2.

---

```

/* Time loop */
for (int t = 0; t < T; ++t) {

    /* Synapse update */
    for (int i = 0; i < N; ++i) {
        float input_current = 0.0f;
        input_current += ext_spikes[t*N + i] *
            ext_weight;
        for (int j = 0; j < N; ++j) {
            input_current += Wflat[i*N + j] *
                s_prev[j];
        }
        g[i] = alpha * g[i] + input_current;
    }
    /* Neuron Update */
    for (int i = 0; i < N; ++i) {
        u[i] -= s_prev[i] * theta;
        u[i] = beta * u[i] + (1.0f - beta) * g[i];
        s[i] = (u[i] > theta);
    }
    /* Copy spikes */
    for (int i = 0; i < N; ++i) {
        s_prev[i] = s[i];
    }
}

```

---

Fig. 1: Time loop of the CPU implementation

This way, the model is expected to be run exactly the same. Meanwhile, as Figure 3 shows, membrane potentials ( $u[t]$ ) and spikes ( $s[t]$ ) are being saved in every timestep in both versions. At the end, those saved values are compared with a script to check the correctness of the C implementation.

---

```

void loadw_f32(const char *fn, size_t count, float
arr[]) {
    FILE *f = fopen(fn, "rb");
    if (!f) {
        printf("File couldn't opened \n");
    }
    float *buf = malloc(sizeof(float) * count);
    size_t r = fread(buf, sizeof(float), count, f);
    if (r != count) fprintf(stderr, "read %zu of %zu
elements\n", r, count);
    fclose(f);
    memcpy(arr, buf, sizeof(float) * N * N);
    printf("Weights are loaded \n");
}

void load_ext_spikes(const char *fn, uint8_t arr[]) {
    uint8_t *buf = malloc(T * N);
    FILE *f = fopen(fn, "rb");
    if (!f) {
        printf("File couldn't opened \n");
    }
    fread(buf, 1, T * N, f);
    fclose(f);
    memcpy(arr, buf, sizeof(uint8_t) * T * N);
    printf("Ext spikes are loaded \n");
}

```

---

Fig. 2: File load functions

Comparison output is given in the figure 4. As we can see from the given figure, the difference is pretty close to zero, with a little error margin because of the difference in float calculations.

```

    #if TEST == 1
    for (int i = 0; i < N; ++i) {
        u_trace[t * N + i] = u[i];
        g_trace[t * N + i] = g[i];
        s_trace[t * N + i] = s[i];
    }
    #endif

    // Outside of the timestep loop
    #if TEST == 1
    FILE *f_u = fopen("u_c.bin", "wb");
    FILE *f_g = fopen("g_c.bin", "wb");
    FILE *f_s = fopen("s_c.bin", "wb");
    fwrite(u_trace, sizeof(float), T * N, f_u);
    fwrite(g_trace, sizeof(float), T * N, f_g);
    fwrite(s_trace, sizeof(uint8_t), T * N, f_s);
    fclose(f_u);
    fclose(f_g);
    fclose(f_s);
    #endif

```

Fig. 3: Saving the traces and to the files

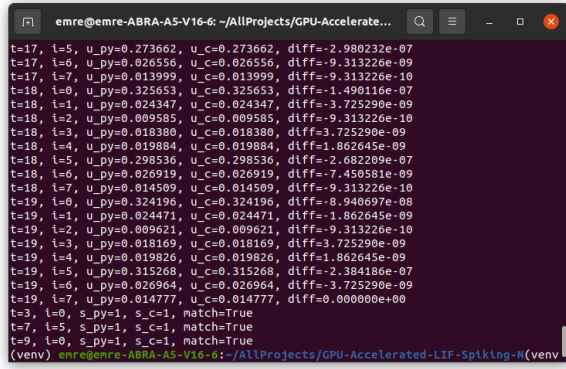


Fig. 4: Comparison output of CPU and pytorch versions

#### IV. BASIC GPU MODEL

##### A. Implementation

The Basic GPU Model implementation is done after the CPU model, to have a middle ground before optimizing this version.

For this basic version, the kernels are separated. One is for the synapse calculation, the other is for the neuron.

The synapse kernel is given at the Figure 5. With the GPU version it is clearly visible that now there is only one for loop, compared to two for loops of the CPU model. Still the biggest bottleneck is this the recurrent calculation loop.

The neuron kernel is given at the Figure 6. After the kernel indexing is done, the membrane potential being updated and the spikes are being recorded.

In this naive version, there are a couple of problems that are apparent. There is two separate kernel introduces an overhead. And the inner loop of the synapse kernel reads the full  $s\_prev$  for every neuron. But the biggest problem is that the Weights are being loaded without a

```

__global__ void synapse_kernel(...)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= n) return;

    // Accumulation
    float acc = (float)ext_step[i] * ext_w;

    // Adding recurrent input from all previous spikes.
    int row = i * n;
    for (int j = 0; j < n; ++j)
        acc += W[row + j] * (float)s_prev[j];

    // Decay
    g[i] = alpha * g[i] + acc;
}

```

Fig. 5: Synapse Kernel of the Basic GPU implementation

```

__global__ void neuron_kernel(...)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= n) return;

    // Membrane Update (Soft Reset)
    float ui = u[i] - (float)s_prev[i] * thresh;
    ui = beta * ui + (1.0f - beta) * g[i];

    // Spike generation
    uint8_t si = (ui > thresh) ? 1 : 0;
    u[i] = ui;
    s[i] = si;
}

```

Fig. 6: Neuron Kernel of the Basic GPU implementation

coalesced access. The weight access is done with  $W[i*n + j]$ , can be seen in the Figure 5. Consecutive threads are not accessing the consecutive memory addresses. This introduces the biggest bottleneck.

##### B. Testing

Testing for the basic GPU version is done with another script; at this step, a Python grapher script has been written to demonstrate the neuron parameters with respect to the timesteps. Three different versions of the same SNN model with the same weights and external spikes are being compared: Python (torch), CPU, and GPU versions.

The best way to ensure that the SNN computations are correct and align with the Python (Torch) reference is to compare the membrane potential with respect to timesteps. In Figure 7, spikes are happening in the correct timesteps and the membrane potentials are aligned perfectly.

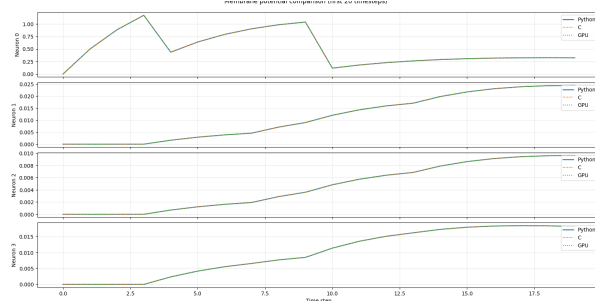


Fig. 7: Membrane Potential comparison of Torch, CPU and GPU versions

The next step is to show the absolute error of the GPU, how much it diverges from the reference. In Figure 8 absolute error of the first eight neurons being shown. The errors are zero for most of the neurons and almost zero for others; this small difference could be the result of the float representation difference between different systems.

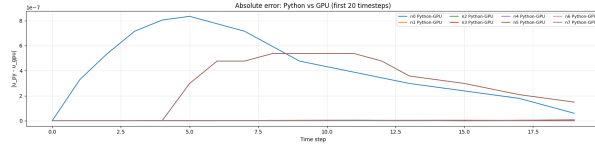


Fig. 8: Absolute error between the Torch and GPU versions

## V. OPTIMIZED GPU MODEL

### A. Implementation

The performance problems of the basic GPU version are previously mentioned in the section IV-A.

The two kernel design choices were for reducing complexity. These two kernels are fused together to optimize the code. By fusing these two kernels into one, the kernel launch overhead at every timestep was reduced to half. Moreover, the  $g_i[t]$  Postsynaptic current was written to global memory and then read by the next kernel at every timestep. With the fusion, the  $g_i[t]$  lives in the registers, and reduces the latency.

Second problem, reading the full  $s\_prev[]$  array (previous timestep spike array) for every neuron. All neurons use the same data, so with shared memory tiling, the data is being loaded into the shared memory to be used by other threads in the block, making this process faster.

Lastly the biggest bottleneck, the un-coalesced access of the weight matrix. The optimal way would be preprocessing the weight matrix with a transpose, and having the access with  $W[j*n + i]$ . But in this project the focus is using GPU optimization techniques, thus float4 vectorized load is introduced.

In the Figure 9, the fused kernel is shown with the mentioned optimizations: fused kernel, memory tiling and vectorized load. The kernel first computes the external contribution of the external spikes and the weights. Then, with the memory tiling and the vectorized load the recurrent connections are accumulated into the acc variable. The tile size was decided as 256, and the block size was also decided as 256; this way, all the threads are loading the data from the global memory to the shared memory tile. In the figure 10, this thread and block count calculation is shown. Lastly, the decay and the membrane potential update are done.

```
__global__ void fused_snn_kernel(...)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    __shared__ uint8_t s_tile[TILE_W];

    float ext_contrib = (i < n) ? (float)ext_step[i] *
        ext_w : 0.0f;

    float acc = ext_contrib;

    const float* W_row = (i < n) ? W + (size_t)i * n :
        NULL;

    for (int tile_start = 0; tile_start < n;
        tile_start += TILE_W) {
        int j_load = tile_start + threadIdx.x;
        s_tile[threadIdx.x] = (j_load < n) ?
            s_prev[j_load] : 0;
        __syncthreads();

        if (i < n) {
            int tile_end = min(TILE_W, n - tile_start);
            int k = 0;
            int tile_end4 = (tile_end / 4) * 4;
            for (; k < tile_end4; k += 4) {
                float4 w4 = *reinterpret_cast<const
                    float4*>(W_row + tile_start + k);
                acc += w4.x * (float)s_tile[k];
                acc += w4.y * (float)s_tile[k + 1];
                acc += w4.z * (float)s_tile[k + 2];
                acc += w4.w * (float)s_tile[k + 3];
            }
        }
        __syncthreads();
    }

    if (i >= n) return;

    float gi = alpha * g[i] + acc;
    g[i] = gi;

    float ui = u[i] - (float)s_prev[i] * thresh;
    ui = beta * ui + (1.0f - beta) * gi;

    uint8_t si = (ui > thresh) ? 1 : 0;
    u[i] = ui;
    s[i] = si;
}
```

Fig. 9: Fused Kernel of the Optimized GPU implementation

### B. Testing

The plotter script expanded to include the optimized version in the plots as well. Ensuring that the optimized

---

```
int threads = TILE_W; /* == 256 */
int blocks = (N + threads - 1) / threads;
```

---

Fig. 10: Thread and Block calculations

version performs with the same correctness. The Figure 11 shows the membrane potential of all the models. Lastly with the Figure 12 it is proven that the optimized GPU version is also aligned with the python (torch) reference.

Lastly, to ensure that the optimization methods are successful, In the Figure 13, the calculation time of the basic and the optimized versions are shown. The basic version finished the calculations in 716ms, the optimized version in 216ms. Thus, the speed up is around 3.3 times.

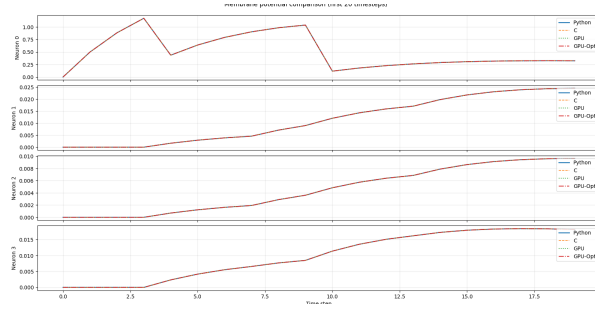


Fig. 11: Comparison of the Membrane Potential between all models.

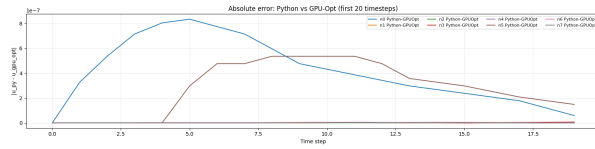


Fig. 12: Absolute error between Optimized GPU and Python.

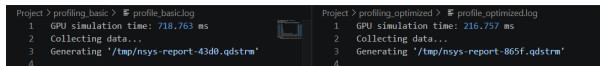


Fig. 13: First comparison of the basic and optimized versions

## VI. RESULTS AND PERFORMANCE EVALUATION

First of all, the sbatch script was edited to be an automated one to see the performance with different N (neuron) and T(timestep) values. And the N and the T variable names are changed into GRID\_N and STEPS\_T. This is done in order to fix the problem with the usage of the nvcc -D flag. The sbatch is compiling the source file as given in the Figure 14. With the automated script,

different N values are given to the compiler to overwrite the defined values. At the end, all the speed results are saved into a file.

---

```
nvcc -O3 -DGRID_N=$n -DSTEPS_T=$t
gpu_optimized.cu -o snn_benchmark.out
```

---

Fig. 14: NVCC compile command

With a Python script, the speed result file is read, and the speed values with different T and N values are plotted as can be seen in the Figure 15. Except for the two divergent points in the graph, the results are expected. The speed-up is eminent in the beginning and slowly decreases while the scaling increases. And the points nearly merge at the highest Neuron Value.

The merge at the end is the result of the memory bandwidth limit. As I mentioned before, the biggest bottleneck is the uncoalesced access to the weight matrix. This is being optimized with the float4 vectorized load. But this only made the transitions more efficient, loading 16 bytes in just one transaction. Basically, using the memory bandwidth more efficiently, But when the neuron count increased, the block count also increased. When N is 256, there is only 1 block; when N is 16384, there are 64 blocks which does parallel transaction requests to/from memory. Which saturates the memory bandwidth without the need for vectorized efficient loads.

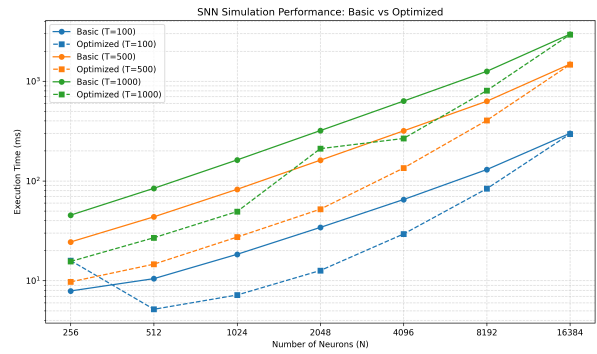


Fig. 15: Execution time with different T and N values

More detailed view of the different neuron counts with a constant timestep is given in the Figure 16. Moreover, the speed factor is given in the Figure 17, which shows that with the scaling, the model becomes bandwidth bound.

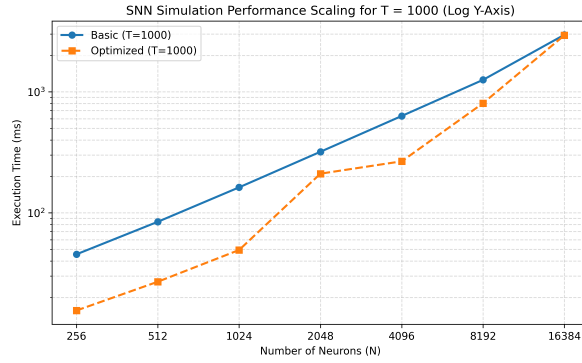


Fig. 16: Execution time with a constant T.

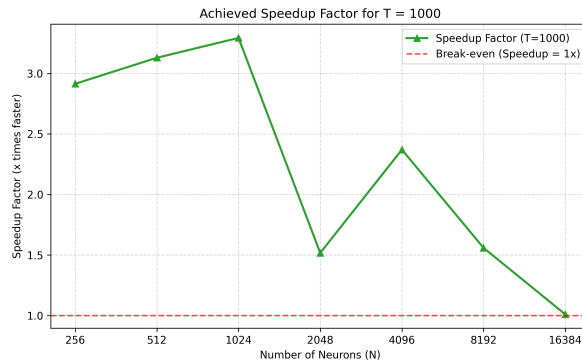


Fig. 17: Speed factor with a constant T.

After the tests comparing the basic and optimized versions of the GPU, comparisons for all the models are planned. First of all, the CPU version is run in the Linux(Ubuntu) environment. This was a necessity to measure the time because in Windows, it is not possible to measure a C code's elapsed time this exact. With the usage of the Linux system calls given in the Figure 18. The C code execution time was measured with different N and T values as well.

```
clock_gettime(CLOCK_MONOTONIC_RAW, &start);
clock_gettime(CLOCK_MONOTONIC_RAW, &finish);
```

Fig. 18: Clock system calls

Then the last model, which is the reference Torch implementation, was measured. This time, the performance counter library is used, the command given in the Figure 19.

At the end, these two benchmark data are also added to the graphs; it is better to note that the CPU that the code runs with has a different, probably a lower clock frequency and performance. Nevertheless, the graphs

```
time.perf_counter()
```

Fig. 19: Performance counter command

provide a good visualization of the speed-up and the performance bottlenecks.

The Figure 20 shows all models' execution time results with different N values. To achieve a more clear visualization the T (timestep) has decided as 1000. As we can see from the graph, it is apparent that the the speed-up achieved with the GPU models. Especially when the tests are scaled up the GPU achieved better speed-up.

When analyzing the two newly tested models, we can see that Torch is much faster than the CPU implementation. CPU is a linear curve as expected, but the Torch is not, even though it is a CPU implementation. This is because the Torch is using optimized CPU instructions and can achieve some level of parallelism. This situation is visible in the first three steps of N in the graph (256, 512, 1024), the CPU's execution time scaled up as expected. While the Torch model is able to stay nearly the same execution duration even though the scaling up. But after the N = 1024 point, the torch's duration also followed the scale-up.

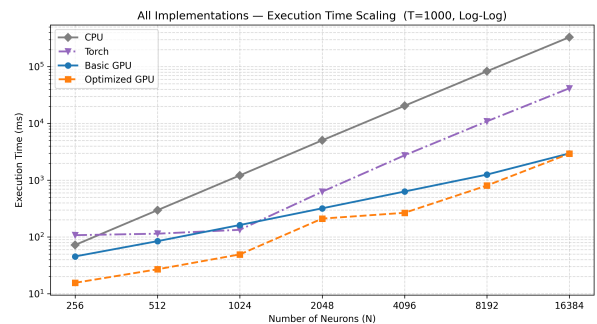


Fig. 20: Execution time of all the models with different N values.

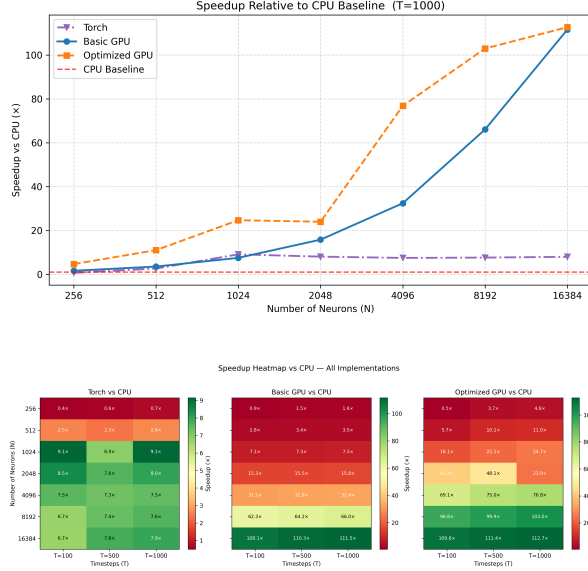


Fig. 21: All models speed up with CPU Baseline, graph and heatmap

In Figure 21, the CPU chosen as a baseline and speed-up is shown. At the above graphical and at the below heatmap visualization are given. In the heatmap visualization, the different T(timestep) values are also shown. When the number of neurons is small, like 256, the CPU generally achieves a better result. One of the reasons for this is that the kernel overhead for GPU parallelism is not being exploited efficiently.

Also the GPU achieved the highest speed up with  $T = 1000$  and  $N = 16384$ . At this point the Optimized GPU version is 112.7 times faster compared to the CPU. Which is not that higher than the Basic version being 111.5 times. This is because the optimizations performed better in the beginning and the middle region of the configurations. This optimizations lost its effect at the late region because the model hit the roof of the memory bandwidth.

## VII. DISCUSSION

The results show a big speed-up with the optimized GPU Model over the CPU model. The speed-up can reach 112.7x when the parameters saturate the GPU. But this big speed-up is not visible across the entire parameter space. Especially when the neuron count is minimal ( $N=256$ ), the GPU setup and kernel launch overheads, makes the GPU slower than the CPU counterpart.

### A. Analysis of Optimization Techniques

Three optimization techniques is introduced for a higher efficiency and achieve more optimized code.

- **Kernel Fusion:** To prevent the kernel launch overhead, kernel fusion is being used. The two kernels

of the Neuron and Synapse calculations merged into one. This also resulted in a lower global memory R/W as the  $g_i[t]$  started to live in the registers.

- **Shared Memory Tiling:** The basic GPU version was reading the whole  $s_{prev}$  array for every neuron. By loading this array in the shared memory tile, the data can be read by all threads in the same block.
- **Vectorized Loads:** The vectorized load of float4 helped the optimized version to use the memory bandwidth more efficiently resulting a faster execution in the beginning and the mid range of parameters.

### B. Memory Bandwidth Bottleneck and Saturation Point

The Optimized and the Basic versions converge at the high number of neurons ( $N = 16386$ ), it is visible from the Figure 20. This is expected as when  $N = 16386$ , the recurrent array of the weights is 1.07 GB.

$$\text{Memory} = 16384^2 \times 4 \text{ bytes} \approx 1.07 \text{ GB}$$

At this point, the simulation is no longer compute-bound; it is bandwidth-bound. From the Figure 17, it is understandable that optimization techniques lost their edge when the execution becomes memory-bandwidth bound.

### C. Matrix Transposition and Sparsity

The introduction of the vectorized loads did help to achieve more efficient memory transfers for low to mid-range. But the real problem is that the uncoalesced memory alignment of the weight array. This is not solved by the vectorized loads. To achieve a more optimized version, the matrix needs to be transposed. Thus enabling consecutive threads to access consecutive weights. At the end, this solution was not implemented to stay on topic.

Another optimization point would be introducing sparsity. Could enable more optimized memory access, changing the  $O(N^2)$  to  $O(E)$ , where E is the number of non-zero synaptic connections. But I did not implemented because of the complexity it would bring, and it is off-topic.

### D. Related Works

CARLsim6 [2], and GeNN [3] are some of the most popular GPU-accelerated simulators. The current state of this work is well below what those simulators are achieving, but still a solid proof-of-concept.

The CARLsim6 is a biologically detailed and large-scale SNN simulator, which is being used in biologically plausible simulations. It provides a C/C++ API, with it user can manage neuron groups and the connectivity patterns. This makes it highly generic and can be used on the majority of the GPUs.

The GeNN is taking another approach; instead of giving a run-time framework, it provides a code generation tool. The user supplies the parameters and the model definitions, and the generator outputs an optimized code for the specific GPU. Their results show a speed-up of up to 200x.

When analyzing the related works, It is apparent that they are highly generic, even though their approach is different.

My work shows a solid proof-of-concept. For future work, the simulator can be more generic. By using the proven optimization techniques given in the report, a Python code generator script can be implemented. The code generator can take the parameters (weights, connectivity, etc.) and generate an optimized code. The GeNN is not specific to Spiking Neural Networks, so optimizations can exploit this specificity.

The NIR (Neuromorphic Intermediate Representation) [4] It is an open-source standard and graph-based format that can provide a graph with the needed neuron parameters. Thus, while feeding the parameters into the code-generator script, the NIR can be used.

#### REFERENCES

- [1] Matei-Ioan Stan and Oliver Rhodes. Learning long sequences in spiking neural networks. *Scientific Reports*, 14(1):21774, 2024.
- [2] L. Niedermeier, K. Chen, J. Xing, A. Das, J. Kopsick, E. Scott, N. Sutton, K. Weber, N. Dutt, and J. L. Krichmar. Carlsim 6: An open source library for large-scale, biologically detailed spiking neural network simulation. In *Proceedings of IEEE International Joint Conference on Neural Networks (IJCNN)*, pages 1–10, 2022.
- [3] Esin Yavuz, James Turner, and Thomas Nowotny. Genn: A code generation framework for accelerated brain simulations. *Scientific Reports*, 6(1):18854, 2016.
- [4] Jens E. Pedersen, Steven Abreu, Matthias Jobst, Gregor Lenz, Vittorio Fra, Felix Christian Bauer, Dylan Richard Muir, Peng Zhou, Bernhard Vogginger, Kade Heckel, Gianvito Urgese, Sadasivan Shankar, Terrence C. Stewart, Sadique Sheik, and Jason K. Eshraghian. Neuromorphic intermediate representation: A unified instruction set for interoperable brain-inspired computing. *Nature Communications*, 15(1):8122, 2024.